DSAA3073 · Theories in Data Science

AdaBoost: reweight data and combine classifiers

Learning Sheet · Lecture 8

Weighted observations, weighted votes, and a training guarantee

A new weighting asks a new question

Suppose a simple classifier makes a few mistakes on a training dataset. Running the same classifier again does not correct them. If we give repeated copies equal votes, they repeat the same predictions. Combining classifiers helps only when the new fits contribute something useful to the current combination.

AdaBoost changes the learning task between rounds. The observations and their labels stay fixed. What changes is how much each observation contributes to the next fitting problem. The new classifier receives a weighted version of the same data, and its predictions are added to a growing vote.

LESSON BACKBONE

How can changing the training weights turn repeated weak fits into a voting rule with a training-error guarantee?

We first identify what a weak-learning promise says. We then connect the changing weights to a loss, derive one round, and follow two rounds on fixed observations. The same calculation will give a training-error bound. Finally, we separate that bound from evidence about new observations.

Throughout this lesson, labels and base predictions are binary: $y_i \in \{-1, +1\}$ and $h(x) \in \{-1, +1\}$. A **base classifier** is one classifier supplied to the voting procedure. We use the term to distinguish it from the combined classifier. It need not be a particular type of tree or linear model.

A population promise and an observed success are different

The earlier PAC framework asked whether a learner could obtain a small population error from enough independent training observations. Recall that population error averages over a new observation. A PAC probability statement averages over the random training sample that produces the learned classifier. These two sources of randomness have different roles.

For the definitions below, let $\mathcal{H}$ be the target hypothesis class. Let $\mathcal{D}$ be any distribution of inputs and let f be a labeling function taking values in $\{-1, +1\}$. The setting is **realizable** when some member of $\mathcal{H}$ agrees with f with probability one under $\mathcal{D}$. Write

$$R_{\mathcal{D},f}(h) = \Pr_{X \sim \mathcal{D}}[h(X) \neq f(X)]$$

for the population classification error. This is the earlier population-risk idea, now written with the input distribution and labeling function explicit.

Definition: Weak learnability

Fix an advantage $\gamma \in \left(0, \frac{1}{2}\right)$. A class $\mathcal{H}$ is **γ-weakly learnable** if there exist a learning algorithm A and one sample-size function $m_{\mathcal{H}}(\delta)$ with the following property.

Choose any $\delta \in (0,1)$, any input distribution $\mathcal{D}$, and any labeling function f for which the setting is realizable. For every size $m \geq m_{\mathcal{H}}(\delta)$, draw m independent inputs from $\mathcal{D}$ and label them by f. Call this random sample S. The guarantee is

$$\Pr\left[R_{\mathcal{D},f}(A(S)) \leq \frac{1}{2} - \gamma\right] \geq 1 - \delta.$$

The probability is over the sample draw and any randomness used by A. The same algorithm and sample-size function must work for all the stated distributions and labeling functions.

The function $m_{\mathcal{H}}$ may depend on the fixed class and advantage, but it cannot be selected separately for each unknown distribution. The guarantee also does not say that every realized sample succeeds. It permits a failure probability at most δ.

Definition: Strong PAC learnability

A class is **strongly PAC learnable** if there exist a learning algorithm A and one sample-size function $m_{\mathcal{H}}(\varepsilon, \delta)$ with the following property.

Choose any accuracy tolerance $\varepsilon \in (0,1)$, confidence parameter $\delta \in (0,1)$, input distribution $\mathcal{D}$, and realizable labeling function f. For every $m \geq m_{\mathcal{H}}(\varepsilon, \delta)$, an independent sample S of that size has the guarantee

$$\Pr\left[R_{\mathcal{D},f}(A(S)) \leq \varepsilon\right] \geq 1 - \delta.$$

Again, the probability includes the sample draw and any learner randomness. The algorithm and sample-size function do not change with the unknown distribution.

The difference is the accuracy request. Weak learning asks for a fixed advantage over error $\frac{1}{2}$. Strong learning allows an arbitrarily small positive tolerance. Both definitions require confidence, realizability, and a distribution-independent sample-size guarantee.

Neither definition alone promises a fast implementation. **Computational efficiency** adds resource requirements, such as polynomial running time under a declared representation and relevant input parameters. A statistical existence statement and a polynomial-resource claim are separate. We will study the mechanism that combines base classifiers and prove its training guarantee; that empirical proof alone will not prove the full population PAC statement.

Put a distribution on fixed training rows

Now fix the observed dataset

$$S = ((x_i, y_i))_{i=1}^{m}.$$

There is no new population sample at each boosting round. Instead, we assign probability masses to its indices. This lets the next base learner treat some observed errors as more important than others.

Definition: Distribution over training observations

At round t, the **example distribution** D_t assigns masses $D_t(i) \geq 0$ to indices $i = 1, ..., m$, with

$$\sum_{i=1}^{m} D_t(i) = 1.$$

It describes emphasis within the fixed training dataset. It is different from the population distribution $\mathcal{D}$. A learner can use these masses directly in a weighted fitting objective or sample training indices according to them.

For example, consider four distinct inputs $x_1, ..., x_4$. Their labels and one supplied base classifier are shown below. The predictions are supplied. This is a constructed trace, not evidence about a particular fitting algorithm.

Observation ID	1	2	3	4
Label y_i	$+1$	$+1$	-1	-1
Prediction $h_1(x_i)$	$+1$	$+1$	$+1$	-1
Initial mass $D_1(i)$	$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$

Only row 3 is wrong. With uniform masses, that error receives one quarter of the total weight. If we later change the masses, the same prediction pattern can have a different weighted error. The labels have not changed; the question asked of the base learner has changed.

Definition: Weighted error and edge

For a base classifier h_t, its weighted training error is

$$\varepsilon_t = \sum_{i=1}^{m} D_t(i) \mathbb{1}[h_t(x_i) \neq y_i].$$

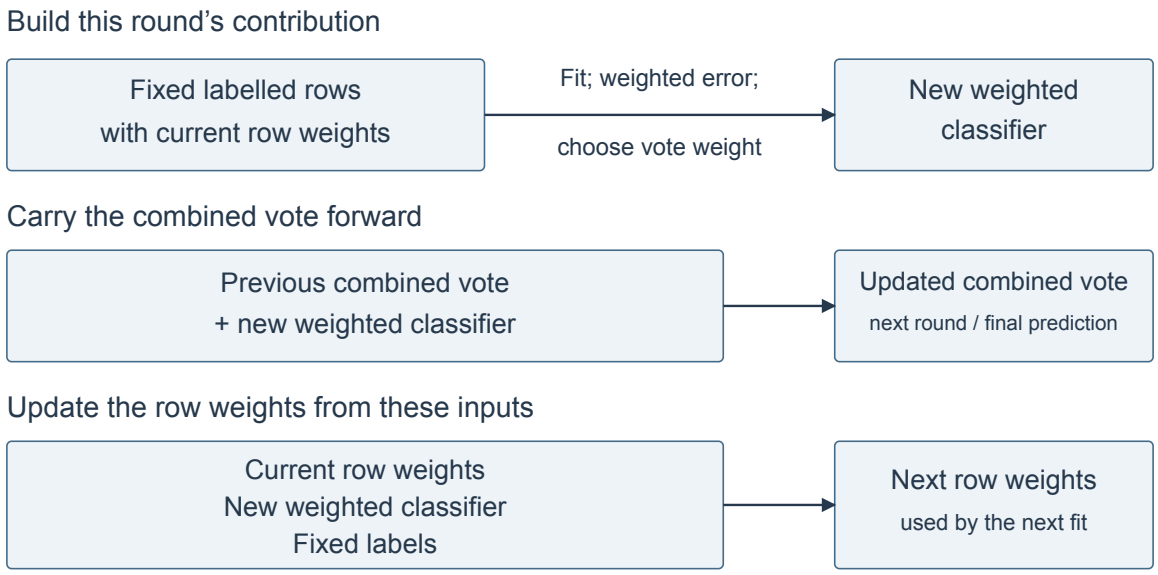
Here $\mathbb{1}[E]$ is one when event E is true and zero otherwise. The **edge** is

$$\gamma_t = \frac{1}{2} - \varepsilon_t.$$

A positive edge means error below one half under the current example distribution. This is an observed property of this round, not the population weak-learnability guarantee.

The supplied first classifier has $\varepsilon_1 = \frac{1}{4}$ and edge $\gamma_1 = \frac{1}{4}$. That observation says nothing by itself about its error on new data or whether a learner will keep finding positive edges after reweighting.

The operational loop is now easy to state: assign row weights, fit a base classifier, measure its weighted error, give it a vote weight, and construct the next row weights. The figure deliberately shows roles before specifying the formulas.



The same new weighted classifier enters both updates. Add its vote to the previous combined vote and carry the result forward. Use the current row weights and fixed labels to form the next row weights. The next round retains the combined vote and fits using the new row weights.

Two quantities must be chosen: the weight of the new classifier in the vote and the new weights of the observations. To choose them coherently, we need a loss that measures what the current vote is doing.

Turn a voting score into a loss

Separate the score from the predicted label

A weighted vote first produces a real number. Taking its sign is a separate operation. Keeping these objects distinct will explain why a loss can change even when the predicted label stays the same.

Definition: Ensemble score and classifier

After t rounds, the **ensemble score** is

$$F_t(x) = \sum_{s=1}^{t} \alpha_s h_s(x), \quad F_0(x) = 0.$$

The scalar α_s is the vote coefficient of base classifier h_s. The combined classifier predicts $+1$ when $F_t(x) \geq 0$ and -1 when $F_t(x) < 0$. We use this fixed $+1$ rule for ties throughout the lesson.

For a labeled observation, $y_i F_t(x_i)$ is its **signed score**. A positive signed score gives the correct label; a negative one gives the wrong label. At zero, the tie rule decides the prediction.

If $y = +1$, a large positive $F(x)$ supports the correct class. If $y = -1$, a large negative $F(x)$ does the same. Multiplying by y puts both cases on one scale. The signed score records direction and strength of the vote, but it is not a calibrated probability of correctness.

For instance, signed scores 0.2 and 2 both give a correct prediction. A count of mistakes treats them alike. A small decrease of 0.3 would reverse the first prediction but not the second. A loss depending on the score can distinguish these situations before a label changes.

Use a smooth loss without changing the target question

A zero-one mistake count is constant while a score moves without crossing its classification boundary. Its derivative therefore does not tell us how to improve those scores. We can instead optimize a smooth loss while retaining a relationship to classification errors.

Definition: Surrogate loss and exponential loss

A **surrogate loss** is a loss optimized in place of the target loss. Here the target is classification error, and the chosen surrogate is the **exponential loss**

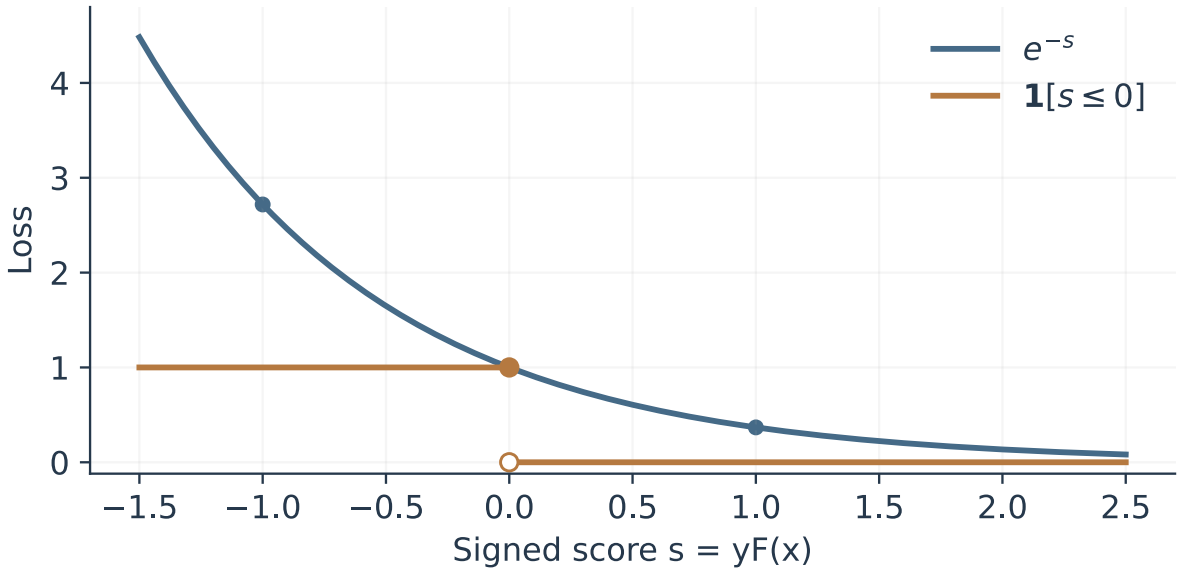
$$\ell_{\exp}(y, F(x)) = \exp(-yF(x)).$$

Its empirical mean on the fixed training sample is

$$L(F) = \frac{1}{m}\sum_{i=1}^{m} \exp(-y_i F(x_i)).$$

We abbreviate $L(F_t)$ as L_t. Since $F_0 = 0$, every initial row loss is one and $L_0 = 1$.

For signed score $s = yF(x)$, the loss is $\exp(-s)$. Negative scores have loss above one, and positive scores have loss below one. The function is smooth and convex. These properties make a one-variable coefficient update manageable, but they do not by themselves provide a convergence rate.



The blue curve distinguishes scores on either side of zero and within the correct region. The amber step counts every nonpositive signed score. Its closed point at $(0, 1)$ *makes that conservative tie convention explicit.*

The graph compares the exponential loss to $\mathbb{1}[s \le 0]$. This indicator counts all zero scores as unfavorable, even though our classifier correctly predicts a positive label at a tie. That makes it a convenient conservative quantity for an upper bound.

Result: Exponential loss bounds classification error

For the fixed tie rule, at each labeled observation,

$$\mathbb{1}[\hat{y} \neq y] \leq \mathbb{1}[yF(x) \leq 0] \leq \exp(-yF(x)),$$

where $\hat{y}$ is the label predicted from the sign of $F(x)$.

If the signed score is positive, both indicators are zero. If it is negative, the exponential is greater than one. If it is zero, the conservative indicator and exponential both equal one, while the actual prediction may be correct or wrong.

Averaging preserves these inequalities. Thus a small mean exponential loss gives an upper bound on training error. The reverse need not hold: a correct prediction can still contribute a substantial positive exponential loss. We will use the upper bound after showing how the algorithm changes the mean loss.

The distinction is visible in supplied values. At signed scores $-1, 0, 1$, exponential losses are approximately $2.718, 1, 0.368$. Moving a correct row from score 0.2 to score 2 reduces its loss from about 0.819 to 0.135, while its zero-one error remains zero. This is why tracking a real loss can reveal progress that a mistake count does not show.

Convert row losses into the next learning emphasis

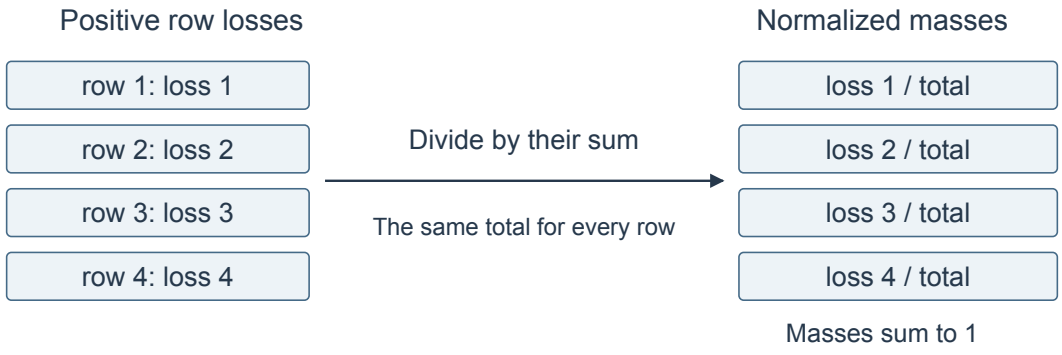
At the start of round t, the current score is F_{t-1}. Its row losses are positive numbers:

$$w_i = \exp(-y_i F_{t-1}(x_i)).$$

The symbol w_i is a temporary name for an unnormalized row loss. It is not a vote coefficient. To make these numbers into a distribution over observations, divide every one by their common sum:

$$D_t(i) = \frac{w_i}{\sum_{j=1}^{m} w_j} = \frac{\exp(-y_i F_{t-1}(x_i))}{\sum_{j=1}^{m} \exp(-y_j F_{t-1}(x_j))}.$$

The denominator equals mL_{t-1}, because L_{t-1} is a mean over m observations. This relation will matter later. A sum of losses, a mean loss, and a distribution are different objects, even though they contain the same row terms.



"Loss 1" means the loss of row 1, not a numerical value. Every row is divided by the same total. Normalization changes the scale and preserves the ratio between any two row losses.

At the initial score $F_0 = 0$, all $w_i = 1$. The sum is m, so $D_1(i) = \frac{1}{m}$. Uniform initialization therefore follows from the zero initial vote. No row has yet received greater emphasis.

Later, a row with a smaller signed score has a larger exponential loss and hence a larger mass. This includes wrongly classified rows and correctly classified rows with small positive scores. The weighting reflects the current combined score, not just the most recent classifier's mistakes.

We now have a concrete rule for the next fitting task: ask the base learner to do well under the normalized losses of the current vote. It remains to determine how much of the chosen classifier to add and how that addition changes these weights.

Derive one complete round

Hold the new classifier fixed and choose its coefficient

Suppose the current score is $F = F_{t-1}$ and the base learner has returned a classifier $h = h_t$. For now, hold both functions fixed. We vary only a real coefficient a in the new score $F + ah$. This is a one-variable optimization problem, not a search over every possible classifier.

For one row, the new exponential loss factors as

$$\exp(-y_i(F(x_i) + ah(x_i))) = \exp(-y_i F(x_i)) \exp(-a y_i h(x_i)).$$

The first factor is the old row loss. The second describes the change caused by the new classifier. Since labels and predictions are binary, $y_i h(x_i)$ equals $+1$ on a correct base prediction and -1 on an incorrect one.

Consequently, every correctly predicted row receives the factor $\exp(-a)$, while every incorrectly predicted row receives $\exp(a)$. The values within each group may differ, but their factors are shared. This is why the weighted error is enough to determine the best coefficient along this particular classifier.

Using $\exp(-y_i F(x_i)) = mL(F)D_t(i)$ gives

$$L(F + ah) = \frac{1}{m} \sum_{i=1}^{m} mL(F)D_t(i) \exp(-a y_i h(x_i)) = L(F) \sum_{i=1}^{m} D_t(i) \exp(-a y_i h(x_i)).$$

Let $\varepsilon = \varepsilon_t$ be the weighted error of this chosen classifier. The incorrect group has total mass ε, and the correct group has total mass $1 - \varepsilon$. Splitting the sum therefore yields

$$L(F + ah) = L(F)[(1 - \varepsilon) \exp(-a) + \varepsilon \exp(a)].$$

The current loss $L(F)$ is a fixed positive factor. Minimizing the new loss is equivalent to minimizing the **loss ratio**

$$g(a) = (1 - \varepsilon) \exp(-a) + \varepsilon \exp(a).$$

At $a = 0$, the score does not change and $g(0) = 1$. A value below one means that adding the classifier reduces the mean exponential loss.

Use calculus to balance the two groups

Differentiate with respect to the coefficient:

$$g'(a) = -(1 - \varepsilon) \exp(-a) + \varepsilon \exp(a).$$

For $0 < \varepsilon < 1$, setting this derivative to zero gives

$$\varepsilon \exp(a) = (1-\varepsilon)\exp(-a),$$

$$\exp(2a) = \frac{1-\varepsilon}{\varepsilon}, \quad a = \frac{1}{2}\log\left(\frac{1-\varepsilon}{\varepsilon}\right).$$

The second derivative is

$$g''(a) = (1-\varepsilon)\exp(-a) + \varepsilon\exp(a) > 0.$$

Thus the stationary point is the unique finite minimum. Both groups have positive mass, so increasing the coefficient cannot reduce both contributions forever: one decreases while the other increases.

Definition: AdaBoost vote coefficient

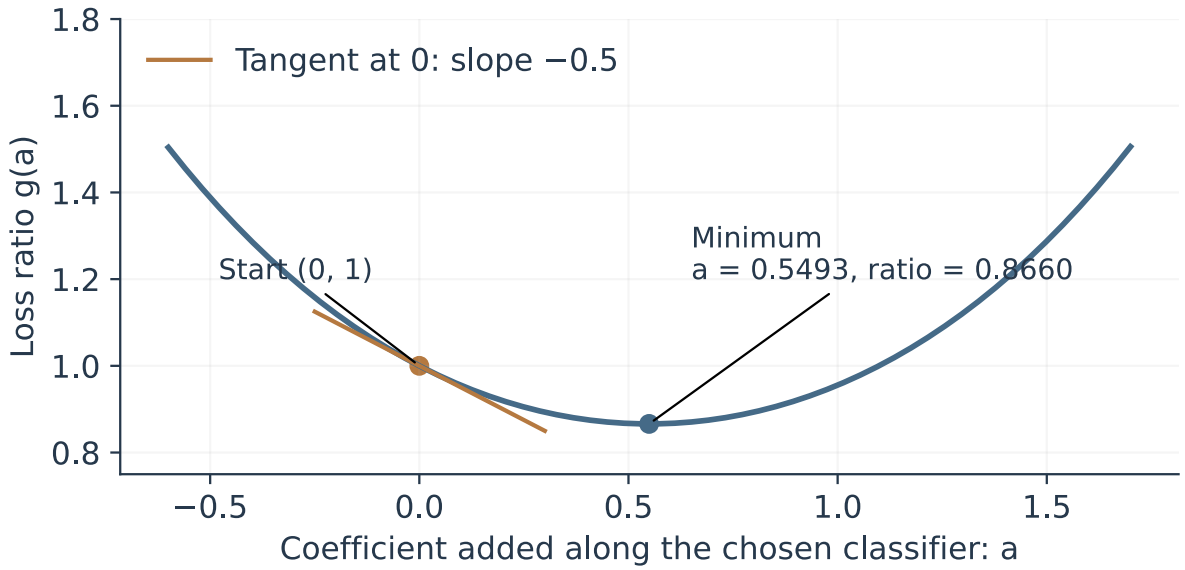
For a chosen classifier with $0 < \varepsilon_t < \frac{1}{2}$, AdaBoost assigns the coefficient

$$\alpha_t = \frac{1}{2}\log\left(\frac{1-\varepsilon_t}{\varepsilon_t}\right).$$

It is positive and minimizes the exponential loss along the score change $F_{t-1} + ah_t$. The word "minimizes" refers to this fixed classifier and current score.

A lower weighted error gives a larger positive coefficient. When the error approaches one half, the coefficient approaches zero. The dependence is logarithmic; the coefficient is not exactly proportional to the edge.

For the first classifier in our four-row example, $\varepsilon_1 = \frac{1}{4}$. Its coefficient is $\alpha_1 = \frac{\log 3}{2} \approx 0.54931$. We can see both the reason for a positive step and the location of its minimum on the actual curve.



Here $g(a) = 0.75\exp(-a) + 0.25\exp(a)$. Its negative slope at zero means a small positive coefficient reduces loss. Continuing past the marked minimum increases the loss again. The plot concerns one chosen classifier with error $\frac{1}{4}$.

The derivative at zero is $g'(0) = 2\varepsilon - 1$. Positive edge therefore gives a negative initial slope. The exact minimizing coefficient goes beyond that local sign check: it balances the two groups at the bottom of the curve.

Substitute the minimizing coefficient into the two group contributions. Since

$$\exp(-\alpha_t) = \sqrt{\frac{\varepsilon_t}{1-\varepsilon_t}}, \quad \exp(\alpha_t) = \sqrt{\frac{1-\varepsilon_t}{\varepsilon_t}},$$

we obtain

$$g(\alpha_t) = (1-\varepsilon_t)\sqrt{\frac{\varepsilon_t}{1-\varepsilon_t}} + \varepsilon_t\sqrt{\frac{1-\varepsilon_t}{\varepsilon_t}} = 2\sqrt{\varepsilon_t(1-\varepsilon_t)}.$$

Each group contributes the same amount at the minimum. This balanced sum will also be the normalizer in the update.

Normalize the new losses, not just their multipliers

After adding the new vote, the row loss becomes

$$\exp(-y_i F_t(x_i)) = \exp(-y_i F_{t-1}(x_i)) \exp(-\alpha_t y_i h_t(x_i)).$$

The next distribution must normalize these **new** losses. Divide the expression by its sum over rows. The old common total cancels, leaving a formula in terms of the current distribution.

Definition: AdaBoost weight update and normalizer

Define the one-round normalizing factor

$$Z_t = \sum_{j=1}^{m} D_t(j) \exp\big(-\alpha_t y_j h_t(x_j)\big).$$

Then set

$$D_{t+1}(i) = \frac{D_t(i) \exp(-\alpha_t y_i h_t(x_i))}{Z_t},$$

and update the score by $F_t = F_{t-1} + \alpha_t h_t$.

For $0 < \varepsilon_t < \frac{1}{2}$ with the coefficient above,

$$Z_t = (1-\varepsilon_t)\exp(-\alpha_t) + \varepsilon_t \exp(\alpha_t) = 2\sqrt{\varepsilon_t(1-\varepsilon_t)}.$$

The factor Z_t ensures that the new masses sum to one. It is not the sum of the unnormalized current row losses, which was mL_{t-1}. The two totals arise at different stages. We will shortly show that Z_t is also the ratio between successive mean losses.

Before normalization, correct base predictions are multiplied by $\exp(-\alpha_t) < 1$ and incorrect predictions by $\exp(\alpha_t) > 1$. After dividing by Z_t, the factors relative to the old masses are

$$\frac{\exp(-\alpha_t)}{Z_t} = \frac{1}{2(1-\varepsilon_t)}, \quad \frac{\exp(\alpha_t)}{Z_t} = \frac{1}{2\varepsilon_t}.$$

For $0 < \varepsilon_t < \frac{1}{2}$, the first is below one and the second above one. In fact, the new total mass on each of these two groups is one half. This balance concerns correctness of the **new base**

classifier. It does not claim that the combined vote has equal mass on its own correct and incorrect predictions.

The distinction matters because the distribution is also exactly the normalized exponential loss of the whole current vote. The update through h_t is a convenient way to calculate it. It does not discard the earlier votes.

Handle boundary errors before using the logarithm

The finite positive-coefficient formula requires $0 < \varepsilon_t < \frac{1}{2}$. A complete algorithm must say what happens outside that interval.

If $\varepsilon_t = 0$, all positively weighted training observations are correctly classified by h_t. Starting from uniform positive weights and using finite updates keeps every weight positive. Thus this base classifier is correct on every training row. There is no finite minimizing coefficient: $g(a) = \exp(-a)$ keeps decreasing as a increases. We can stop and return this perfect training classifier instead of trying to evaluate a logarithm involving division by zero. This conclusion is about the observed rows, not new data.

If $\varepsilon_t = \frac{1}{2}$, the coefficient is zero and $Z_t = 1$. The score and distribution do not change. Repeating the identical step produces no progress, so stop or ask the learner for another classifier.

If $\varepsilon_t > \frac{1}{2}$, this classifier does not meet the positive-edge condition. Reject it or stop. Negating its predictions would change its error to $1 - \varepsilon_t$, but that option is valid only when the allowed base class contains the negated function. If we use it, recompute the error for the function actually added to the vote.

Result: AdaBoost: the complete finite-round procedure

Inputs: a fixed binary-labeled sample, a base-learning rule that accepts example weights, and a maximum number of rounds T.

1. Set $F_0(x) = 0$ and $D_1(i) = \frac{1}{m}$.
2. At round t, obtain h_t using the training rows weighted by D_t.
3. Calculate its weighted error ε_t. Apply the zero-error, half-error, or above-half policy before the positive-coefficient branch.
4. When $0 < \varepsilon_t < \frac{1}{2}$, calculate α_t, add $\alpha_t h_t$ to the score, calculate Z_t, and normalize to obtain D_{t+1}.
5. After the executed finite rounds, return the sign classifier of the accumulated score, using $+1$ at a tie. The separate zero-error stopping branch returns its perfect training base classifier.

The coefficient and distribution formulas follow from the exponential-loss objective. Other boosting updates may optimize different objectives or need different arguments. To understand this update, follow its effect on the same four rows and distinguish the observation weights from the classifier votes.

Follow the same observations through two rounds

First round: one error receives more emphasis

Keep the four labels and first classifier introduced earlier. All arithmetic below is supplied so that we can concentrate on the changes in the model and distribution.

Worked example: The first round

With $D_1(i) = \frac{1}{4}$ and $h_1 = (+1, +1, +1, -1)$, only row 3 is wrong. Thus

$$\varepsilon_1 = \frac{1}{4}, \quad \alpha_1 = \frac{\log 3}{2} \approx 0.54931, \quad Z_1 = \frac{\sqrt{3}}{2} \approx 0.86603.$$

The new distribution is

$$D_2 = \left(\frac{1}{6}, \frac{1}{6}, \frac{1}{2}, \frac{1}{6}\right).$$

The three correct rows have been multiplied by $\frac{1}{[2(1-\frac{1}{4})]} = \frac{2}{3}$, while the incorrect row has been multiplied by $\frac{1}{2(\frac{1}{4})} = 2$. These are the factors **after normalization**.

The score is $F_1 = \alpha_1 h_1$. Its signed scores are

$$(y_i F_1(x_i))_{i=1}^4 = (\alpha_1, \alpha_1, -\alpha_1, \alpha_1).$$

The combined classifier still makes one error, on row 3. Its mean exponential loss is $L_1 = Z_1 \approx 0.86603$.

The next base learner now sees row 3 carrying half the total mass. A classifier that repeats the same error on row 3 already has error at least one half under D_2. The new weighted task therefore discourages simply returning the first classifier unchanged. It does not force a particular replacement or guarantee that one exists in the chosen base class.

Second round: a stronger vote can move an error

Suppose the learner returns the supplied second classifier

$$h_2 = (-1, +1, -1, -1).$$

It correctly predicts row 3, but it is wrong on row 1. Under D_2, row 1 has mass $\frac{1}{6}$. Therefore the second weighted error is $\frac{1}{6}$, even though both base classifiers make one error when counted uniformly over the four rows.

Worked example: The second round

The calculated coefficient and normalizer are

$$\varepsilon_2 = \frac{1}{6}, \quad \alpha_2 = \frac{\log 5}{2} \approx 0.80472, \quad Z_2 = \frac{\sqrt{5}}{3} \approx 0.74536.$$

The resulting distribution is

$$D_3 = \left(\frac{1}{2}, \frac{1}{10}, \frac{3}{10}, \frac{1}{10}\right).$$

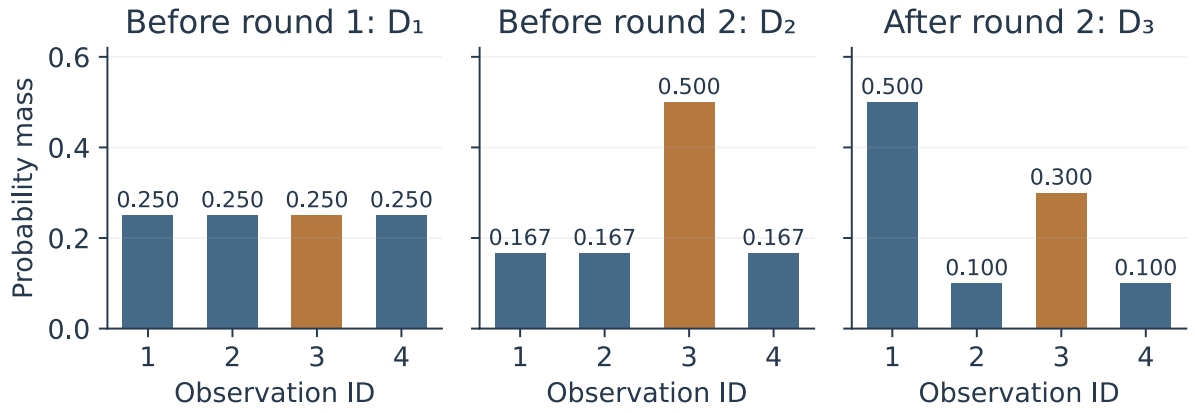
The score is now $F_2 = \alpha_1 h_1 + \alpha_2 h_2$. The completed row comparison is:

Row	y_i	h_1	h_2	Signed score after round 1	Signed score after round 2
1	$+1$	$+1$	-1	0.54931	-0.25541
2	$+1$	$+1$	$+1$	0.54931	1.35403
3	-1	$+1$	-1	-0.54931	0.25541
4	-1	-1	-1	0.54931	1.35403

Since $\alpha_2 > \alpha_1$, the second classifier decides the label where the two classifiers disagree. Row 3 becomes correct and row 1 becomes wrong. The training error remains $\frac{1}{4}$, while the mean exponential loss decreases to

$$L_2 = \frac{\sqrt{15}}{6} \approx 0.64550.$$

The larger second coefficient follows from its lower **weighted** error, not from a lower unweighted error count. The row comparison also shows why "every observation improves" is too strong. Row 1's signed score has decreased through zero. The objective decreases in aggregate, allowing one contribution to worsen while others improve.



All three panels use the same probability-mass scale and observation IDs. Amber always identifies row 3, not "the wrong row." After round 2, row 1 receives the largest mass even though row 3 keeps its color.

Read row 3 across the panels. Its mass rises from $\frac{1}{4}$ to $\frac{1}{2}$ after the first error, then falls to $\frac{3}{10}$ after the second classifier gets it right. It still receives more mass than rows 2 and 4 because the whole vote gives it a smaller positive signed score. Reweighting is therefore more informative than a list of the latest classifier's mistakes.

Now read row 1. Its mass becomes $\frac{1}{2}$ after the second round, when h_2 gets it wrong. This does not mean row 1 contributes half of the final vote. Its mass belongs to the **next fitting problem**. The vote coefficients are α_1 and α_2, attached to classifiers, not observations.

Check the objects without repeating the arithmetic

The two rounds let us compare three different kinds of change.

First, both base classifiers have one unweighted error, yet their weighted errors and coefficients differ. The distribution matters. Second, changing the distribution does not alter any label or

feature value. It alters the fitting objective. Third, the mean exponential loss decreases even though the combined training error remains $\frac{1}{4}$ and one formerly correct row becomes wrong.

These are observations about two specified executed rounds. They do not prove that a base learner will achieve positive edge under every future distribution. To obtain a guarantee after many rounds, we must make the successful-round assumption explicit and connect the observed normalizers to the loss being bounded.

Turn the normalizers into a training-error bound

The same factor normalizes weights and measures progress

The update introduced Z_t to make new masses sum to one. That operational role alone does not show why multiplying such factors should bound an error. We need to connect the factor to the mean loss, keeping the sum-versus-mean distinction explicit.

At the start of round t, the distribution is

$$D_t(i) = \frac{\exp(-y_i F_{t-1}(x_i))}{mL_{t-1}}.$$

At the end, the mean loss is

$$L_t = \frac{1}{m}\sum_{i=1}^{m} \exp(-y_i F_t(x_i)).$$

Substitute $F_t = F_{t-1} + \alpha_t h_t$ and factor each row's exponential:

$$L_t = \frac{1}{m}\sum_{i=1}^{m} \exp(-y_i F_{t-1}(x_i)) \exp(-\alpha_t y_i h_t(x_i)).$$

The old row loss equals $mL_{t-1}D_t(i)$. Substituting that expression gives

$$L_t = \frac{1}{m}\sum_{i=1}^{m} mL_{t-1}D_t(i) \exp(-\alpha_t y_i h_t(x_i)).$$

The factor m cancels, and L_{t-1} is common to every term:

$$L_t = L_{t-1}\sum_{i=1}^{m} D_t(i) \exp(-\alpha_t y_i h_t(x_i)) = L_{t-1}Z_t.$$

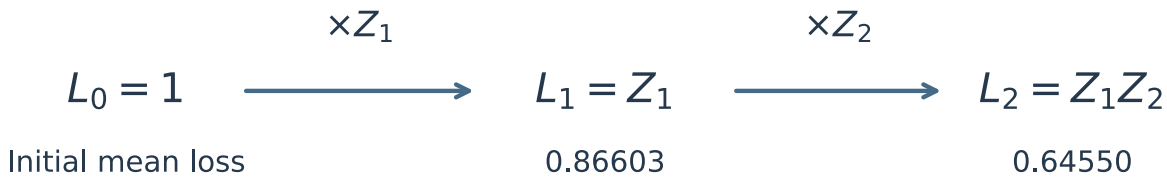
Thus $Z_t = \frac{L_t}{L_{t-1}}$. It is a dimensionless ratio of consecutive mean losses. The same calculation explains why the normalized-weight formula remains valid after updating: dividing the new row losses by $mL_t = mL_{t-1}Z_t$ produces exactly the AdaBoost update. Uniform initialization supplies the base case, and this identity carries the relationship through every finite round.

Result: Exact product of the normalizers

Starting from $F_0 = 0$ and $L_0 = 1$, every executed finite update satisfies $L_t = L_{t-1} Z_t$. Therefore

$$L_T = \prod_{t=1}^{T} Z_t.$$

This equality concerns the mean exponential loss. It is not an equality for the number of classification errors.



Each arrow uses a ratio of consecutive mean losses. In the supplied trace, $Z_1 \approx 0.86603$ *and* $Z_2 \approx 0.74536$. *Their product is the second mean loss, not the training-error fraction* $\frac{1}{4}$.

We can also see the cancellation by writing the ratios directly:

$$\frac{L_T}{L_{T-1}} \frac{L_{T-1}}{L_{T-2}} \cdots \frac{L_1}{L_0} = \frac{L_T}{L_0} = L_T.$$

The intermediate losses cancel. Multiplication is therefore a consequence of the exact recurrence, not an approximation based on a plotted curve.

Bring the classification error into the argument

Let $\hat{y}_T(x)$ denote the final sign prediction with the same $+1$ tie rule. Its empirical error is

$$\hat{R}_S = \frac{1}{m} \sum_{i=1}^{m} \mathbb{1}[\hat{y}_T(x_i) \neq y_i].$$

We already established the pointwise relation between an actual classification error, a nonpositive signed score, and an exponential loss. Average that relation over the fixed training rows:

$$\hat{R}_S \leq \frac{1}{m} \sum_{i=1}^{m} \mathbb{1}[y_i F_T(x_i) \leq 0] \leq \frac{1}{m} \sum_{i=1}^{m} \exp(-y_i F_T(x_i)) = L_T.$$

Combining with the exact product gives

$$\hat{R}_S \leq L_T = \prod_{t=1}^{T} Z_t.$$

There are two different kinds of relation in this line. The error is **bounded by** the mean exponential loss. The mean exponential loss **equals** the product. Our two-round example has error 0.25 and mean loss about 0.64550, so replacing the first inequality by equality would already fail on the supplied data.

The tie convention also matters. At score zero, the conservative indicator counts one for either label. Our classifier predicts +1, so a positively labeled row is actually correct. This is why the nonpositive-score frequency is itself an upper bound rather than an exact expression for our classifier's error.

Translate a positive edge into a factor below one

For a finite successful round, we derived

$$Z_t = 2\sqrt{\varepsilon_t(1-\varepsilon_t)}.$$

Write the edge as $\gamma_t = \frac{1}{2} - \varepsilon_t$. Then

$$\varepsilon_t(1-\varepsilon_t) = \left(\frac{1}{2} - \gamma_t\right)\left(\frac{1}{2} + \gamma_t\right) = \frac{1}{4} - \gamma_t^2,$$

so

$$Z_t = \sqrt{1 - 4\gamma_t^2}.$$

With $0 < \varepsilon_t < \frac{1}{2}$, we have $0 < \gamma_t < \frac{1}{2}$ and $0 < Z_t < 1$. Every such round decreases the mean exponential loss by a multiplicative factor. This conclusion uses the coefficient formula and edge condition, not convexity alone.

To obtain a simpler rate, use the elementary inequality $1 - u \le \exp(-u)$ for $u \ge 0$. One justification is that the convex exponential curve lies above its tangent at zero: $\exp(v) \ge 1 + v$; set $v = -u$. Taking nonnegative square roots at $u = 4\gamma_t^2$ yields

$$\sqrt{1 - 4\gamma_t^2} \le \exp(-2\gamma_t^2).$$

This is an inequality. No small-edge approximation is needed.

Result: AdaBoost training-error bound

Suppose the algorithm executes T finite rounds, each with $0 < \varepsilon_t < \frac{1}{2}$, using the derived coefficients and updates. Let $\gamma_t = \frac{1}{2} - \varepsilon_t$. Then

$$\hat{R}_S \le L_T = \prod_{t=1}^{T} 2\sqrt{\varepsilon_t(1-\varepsilon_t)} \le \exp\left(-2\sum_{t=1}^{T}\gamma_t^2\right).$$

If every round has $\gamma_t \ge \gamma > 0$, this implies

$$\hat{R}_S \le \exp(-2T\gamma^2).$$

The separate zero-error stopping branch already returns a classifier with zero training error; it does not use an infinite coefficient in this finite-round product.

The first version accommodates different edges across rounds. The second is convenient when a common positive lower bound is known. We cannot infer that common lower bound from the first round alone. Later weightings may be harder for the base learner.

If the base-learning algorithm is randomized, the statement conditions on the executed rounds meeting the required errors. A population weak-learning contract includes its own failure probability. An unconditional guarantee for many calls would also have to control those failures.

That probability argument is separate from the deterministic product calculation on a successful trace.

Read a round budget as a sufficient bound

Suppose a common lower edge γ is available at every executed round and the desired training-error bound is $\eta \in (0, 1)$. Solving

$$\exp(-2T\gamma^2) \leq \eta$$

gives the sufficient integer condition

$$T \geq \left\lceil \frac{\log\left(\frac{1}{\eta}\right)}{2\gamma^2} \right\rceil.$$

The following values are supplied for target $\eta = 0.01$.

Lower edge γ	Per-round error at most	Sufficient rounds for the bound to be at most 1%
0.01	0.49	23026
0.05	0.45	922
0.10	0.40	231
0.20	0.30	58

The quadratic dependence on γ is visible: doubling the lower edge divides the unrounded sufficient budget by four. But the table is not a forecast of observed learning curves. A dataset may reach a lower training error earlier, and the required edge may fail later. The values also say nothing about a 1% test error.

The first row corrects a tempting shortcut in reasoning. Being slightly better than one half can support a training guarantee under repeated successful reweighting, but it does not justify a dramatic accuracy claim after an arbitrarily small number of rounds. The edge, number of rounds, update rule, and type of error all belong in the statement.

The updates determine the loss ratios. Positive edges keep those ratios below one, and their product equals the mean exponential loss. That loss bounds the empirical error. The rate follows from this chain of relations, with each assumption used at a specific step.

Understand the step, then evaluate the chosen predictor

View one base classifier as a direction

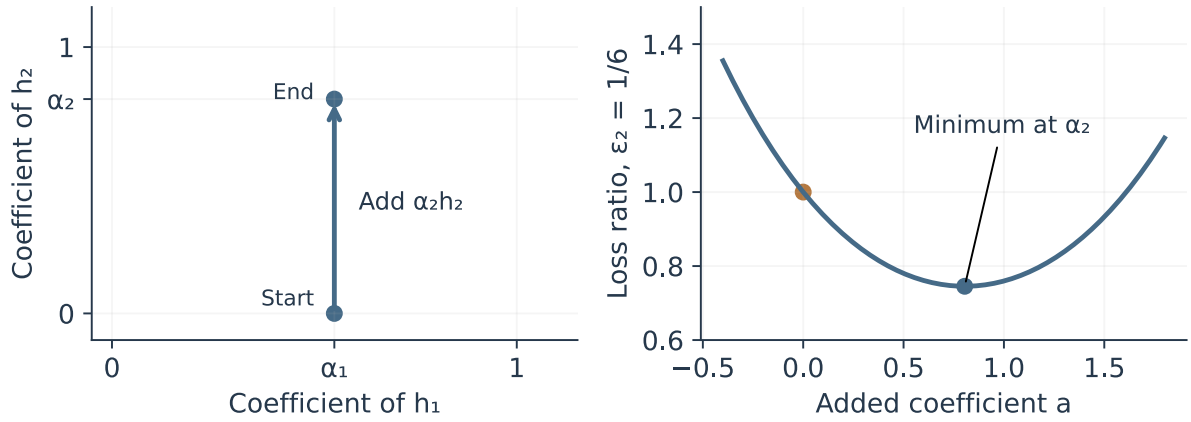
The derivation fixed a classifier h and varied only its coefficient. There is a useful optimization interpretation of that operation. Imagine a fixed collection of base functions $h_1, h_2, \ldots$. A coefficient vector determines a score by adding weighted copies of those functions.

Definition: Coordinate descent

In a coefficient representation, **coordinate descent** changes one coefficient while holding the others fixed. If a score is represented as $F = \sum_j \beta_j h_j$, changing coefficient j by a changes the score to $F + ah_j$.

For a selected base function, minimizing over a is a one-dimensional **line search**. AdaBoost's coefficient calculation performs this exact minimization for the chosen classifier under exponential loss.

The coordinates in this description are coefficients of functions. They are not coordinates of an observed feature vector. Reusing the same base function in a later round adds to its existing coefficient; it does not create a new prediction rule merely by giving the round a new index.



Both panels show the supplied second round. The left moves from $(\alpha_1, 0)$ to (α_1, α_2). The right uses $\varepsilon_2 = \frac{1}{6}$ and reaches its minimum at $a = \alpha_2 \approx 0.80472$. Its horizontal axis is a coefficient change, not a feature value.

For the currently chosen h, write ε for its weighted error under D_t. Our earlier calculation gives

$$\left[\frac{d}{da} L(F + ah)\right]_{a=0} = -L(F) \sum_{i=1}^{m} D_t(i) y_i h(x_i) = -L(F)(1 - 2\varepsilon).$$

The equality $\sum_i D_t(i) y_i h(x_i) = 1 - 2\varepsilon$ follows by subtracting the incorrect mass from the correct mass. Positive edge makes the derivative negative. The base classifier therefore supplies a direction in which a small positive coefficient decreases loss; the logarithmic coefficient finds the minimum along it.

This does not establish that every positive-edge classifier is the steepest available coordinate. That stronger claim depends on how the base learner selects among the entire allowed collection. Our current guarantee requires a positive edge, not an exact global search over all base functions. The distinction preserves the useful interpretation without attributing an extra selection property to the base learner.

Keep optimization, training error, and new-data performance separate

The operational and optimization views now explain the same quantities. Example weights identify the row losses emphasized by the next fit. A vote coefficient is an exact step size along

the chosen classifier. The normalizer is both a probability normalization factor and a ratio of consecutive mean losses.

The training theorem adds another conclusion: under the successful-round assumptions, those ratios produce an upper bound on empirical classification error. A new-data error is a different quantity. The proof summed over the fixed training rows; no independent evaluation sample appeared in it.

A zero training error also does not mean the exponential loss is zero. Correct rows can have different positive signed scores and positive losses. More rounds can therefore change the loss after the labels are already correct. Whether new-data predictions improve is a separate question; the training bound does not answer it.

> **Worked example: Choose a stopping round and evaluate the choice**
>
> Suppose a learner produces candidate ensembles after several declared numbers of rounds. The goal is classification on new observations. Use training data to fit the sequence and validation data to compare the candidate stopping rounds using the chosen prediction loss.
>
> If the validation comparison selects a stopping round, that validation score has helped make the choice. Evaluate the completed selection-and-fitting procedure using separate test evidence or an appropriate outer cross-validation procedure, as in Lecture 7. Any other choices based on validation, such as the base learner's complexity, belong to the same selection stage.
>
> Report the observed training behavior and the independent evaluation result as different evidence. A decreasing exponential training loss explains the optimization progress; it does not authorize replacing the independent prediction score with the training bound.

The sufficient round budget derived above can help interpret a stated lower-edge assumption. It is not a universal stopping rule for prediction. Conversely, validation-based stopping does not alter the algebra of the rounds already executed. The two ideas answer different questions and can be used together when their roles are clear.

Read the whole argument through its main objects

The fixed dataset and labels are the common starting point. The distribution D_t assigns masses to observed rows; it is not the population distribution. The weighted error ε_t measures one base classifier under those masses, and its edge is $\frac{1}{2} - \varepsilon_t$.

The score F_t adds classifier votes. Its sign gives a label, while the signed score $yF_t(x)$ enters exponential loss. Normalizing those losses gives the next example distribution. Minimizing the loss along a chosen classifier gives α_t and the update normalizer Z_t.

The proof then follows the quantities that actually change: $L_t = L_{t-1}Z_t$, hence $L_T = \prod_t Z_t$. Empirical classification error is bounded by this loss, and a positive lower edge gives the exponential upper bound. Exact equalities describe the specified update; inequalities describe what it guarantees.

The final question is therefore concrete: did the learner execute the required successful rounds, and what evidence evaluates the predictor chosen from them? The first question concerns the algorithm and its training loss. The second carries the selection-and-evaluation distinction forward to new observations.